



PRACTICAL INTERVIEW GUIDE

GenAI Interview Questions

Crack AI & GenAI Roles with Confidence

**LLMs &
Transformers**

**Prompt
Engineering**

**RAG &
Vector DBs**

**AI Agents &
LangChain**

**Fine-tuning
& LoRA**

**Evaluation &
Production**



RESNUR AI

Short, precise answers for quick revision →

What is Generative AI? How is it different from traditional AI?

ANSWER

Traditional AI classifies, predicts, or decides based on patterns — discriminative modeling. Generative AI creates new content that didn't exist before — text, images, code, audio.

Simple way I think about it: traditional AI answers "what is this?" Generative AI answers "create something new." A spam classifier is traditional AI. ChatGPT writing an email is generative AI.

DISCRIMINATIVE

GENERATIVE

What is a Large Language Model?

ANSWER

An LLM is a deep learning model trained on massive text data to understand and generate human language. What makes them "large" is scale — billions of parameters trained on internet-scale data using Transformer architecture.

The key capability that emerges at scale is generalization — performing tasks they were never explicitly trained for. GPT-3 has 175B parameters. GPT-4 is estimated around 1 trillion.

SCALE

GENERALIZATION

1 REVISE & PREPARE FOR GENAI ROLES

Explain the Transformer architecture. Why did it replace RNNs?

ANSWER

Transformer was introduced in "Attention Is All You Need" (2017). Core innovation: self-attention — every token attends to every other token simultaneously instead of sequentially.

RNNs had two problems: no parallelization and vanishing gradients causing long-range dependency failures. Transformers solved both — full parallelization and direct token-to-token attention regardless of distance.

Three variants: decoder-only (GPT), encoder-only (BERT), encoder-decoder (T5).

SELF-ATTENTION

VS RNNs

What is the attention mechanism?

ANSWER

Attention decides which parts of the input to focus on when processing each token.

Technically: three matrices — Query ("what am I looking for?"), Key ("what does each token offer?"), Value ("what information does each token carry?"). Attention scores = dot product of Query and Key, softmaxed into weights, applied to Values.

Self-attention means every token attends to every other token. Multi-head attention runs this in parallel — each head learns different relationship types.

Q/K/V MATRICES

MULTI-HEAD

What is a token? How does tokenization work?

ANSWER

A token is the basic unit an LLM processes — roughly word pieces, not full words. "unhappiness" → ["un", "happiness"] = 2 tokens. Rule of thumb: 1 token $\approx$ 0.75 words. 1000 tokens $\approx$ 750 words.

Why it matters: context windows are measured in tokens, API costs are per token, and tasks like counting letters are hard because LLMs operate on tokens not characters.

Most common method: Byte Pair Encoding (BPE) — merges most frequent character pairs iteratively.

BPE TOKENIZER

TOKEN BUDGET

What is a context window?

ANSWER

The maximum text — in tokens — an LLM can process in a single call. Everything outside is invisible to the model.

GPT-3: 4K tokens. GPT-4 Turbo: 128K. Gemini 1.5 Pro: 1M.

Practical impact: conversation history gets truncated, RAG context must fit within the window, long documents need chunking. Important nuance: "lost in the middle" — LLMs perform worse on information buried in the middle of long contexts vs beginning or end.

CONTEXT LIMIT

LOST IN MIDDLE

What is hallucination? How do you reduce it?

ANSWER

Hallucination is when an LLM generates confident but factually wrong information. It happens because LLMs predict statistically likely tokens — optimizing for plausibility, not truth.

Three types: factual (wrong facts), faithfulness (contradicts provided context), temporal (outdated info stated as current).

Mitigation: RAG to ground responses in real documents, lower temperature for factual tasks, instruct model to say "I don't know" when uncertain, output validation against source documents, LLM-as-Judge evaluation.

[HALLUCINATIONS](#)[MITIGATIONS](#)

What is Temperature, Top-P, and Top-K?

ANSWER

All control output randomness during token sampling.

Temperature scales probability distribution. 0 = deterministic (always picks highest probability token). 1 = default. Above 1 = more creative, less coherent. I use 0 for code and factual tasks, 0.7 for conversation.

Top-K limits sampling to K most likely tokens. Top-P (nucleus sampling) samples from the smallest token set whose cumulative probability exceeds P — more adaptive than Top-K.

In practice: tune temperature first, keep Top-P around 0.9 as secondary control.

[SAMPLING](#)[NUCLEUS SAMPLING](#)

Difference between GPT, BERT, and T5?

ANSWER

GPT — decoder-only. Predicts next token left-to-right. Best for generation: chat, completion, code.

BERT — encoder-only. Predicts masked tokens bidirectionally. Best for understanding: classification, NER, semantic search.

T5 — encoder-decoder. Frames everything as text-to-text. Best for seq-to-seq: summarization, translation, QA.

Rule: need to generate → GPT family. Need to classify/understand → BERT family. Need flexible seq-to-seq → T5 family.

[GPT VS BERT](#)[ENCODER/DECODER](#)

What is prompt engineering? Key techniques?

ANSWER

Designing and optimizing inputs to get reliable, high-quality LLM outputs.

Key techniques I use:

- Zero-shot — ask directly, no examples
- Few-shot — 3-5 examples before actual request, improves consistency dramatically
- Chain of Thought — "think step by step," forces reasoning before answering
- System prompts — set behavior, persona, constraints, output format
- Role prompting — "You are a senior security engineer reviewing this code"
- Output formatting — explicitly specify JSON, structure, length

PROMPT ENG

COT / SYSTEM

1 REVISE & PREPARE FOR GENAI ROLES

What is RAG? How does it work?

ANSWER

RAG grounds LLM responses in retrieved external documents instead of relying on training data — solving hallucination and knowledge staleness.

How it works: chunk documents → embed chunks → store in vector DB → at query time, embed query → retrieve top-K similar chunks → pass chunks + query to LLM → LLM generates answer grounded in retrieved context.

Key components: chunking strategy, embedding model, vector database, reranker (optional), LLM generator, evaluation layer.

KNOWLEDGE RETRIEVAL

VECTOR INGESTION

RAG vs Fine-tuning — when do you use each?

ANSWER

RAG is for knowledge. Fine-tuning is for behavior.

Use RAG when: knowledge changes frequently, you need source citations, working with private documents, limited compute budget. Faster to implement, cheaper to maintain, more auditable.

Use fine-tuning when: you need consistent tone/style/format, domain has specialized vocabulary, prompt engineering can't achieve the behavior, latency is critical and shorter prompts help.

Most common mistake: jumping to fine-tuning before exhausting RAG and prompt engineering. Best production systems often combine both.

RAG VS FINE-TUNING

DESIGN CHOICE

What are embeddings?

ANSWER

Dense numerical vector representations where semantic similarity = geometric proximity.

Classic example: "king" - "man" + "woman" $\approx$ "queen" in embedding space.

Generated by passing text through an encoder model and taking output representations — typically mean pooling of token embeddings.

Used for: semantic search, RAG retrieval, clustering, classification, duplicate detection.

Critical rule: always use the same embedding model for indexing and querying — mixing models breaks the semantic space.

DENSE VECTORS

EMBEDDING MODELS

What is a vector database?

ANSWER

Purpose-built to store and query high-dimensional vectors via similarity search. Core operation: given a query vector, find K most similar vectors.

Uses Approximate Nearest Neighbor (ANN) algorithms — typically HNSW — for sub-linear search time instead of exact $O(n)$ comparison.

Similarity metrics: cosine similarity (text), dot product (OpenAI embeddings), Euclidean distance (image embeddings).

My choices: ChromaDB for prototyping, Pinecone for managed production, Qdrant for self-hosted production, FAISS for raw Python performance.

ANN SEARCH

HNSW INDEXING

What is LangChain?

ANSWER

Open source framework providing abstractions for building LLM applications — chains, agents, memory, document loaders, vector store integrations, output parsers.

Core components:

- Chains — sequences of LLM calls via LCEL
- Agents — LLM that picks and uses tools dynamically
- Memory — conversation history management
- Document loaders — 100+ data source integrations
- Output parsers — structure responses into Pydantic, JSON

Honest take: abstractions can be leaky. For simple use cases I sometimes prefer direct LLM calls. For complex multi-step applications it saves significant time.

ORCHESTRATOR

LCEL COMPOSITION

What is LangGraph? When use it over LangChain?

ANSWER

LangGraph builds stateful, graph-based agent workflows on top of LangChain. Key difference: LangChain builds linear chains, LangGraph builds cyclical graphs.

Real agent tasks need loops — try something, check result, retry if needed. LangChain can't express this. LangGraph can.

Core concepts: Nodes (processing steps), Edges (transitions), Conditional edges (branching on state), State (shared typed dictionary), Human-in-the-loop (pause for approval).

Use LangGraph when: tasks require loops/retries, multi-agent collaboration, human approval checkpoints, or complex branching logic.

STATEFUL GRAPH

CYCLICAL LOOPS

What is an AI Agent? How does ReAct work?

ANSWER

An agent is an LLM that plans, acts, observes results, and iterates toward a goal — unlike a chatbot that just responds.

ReAct pattern — Reasoning + Acting:

Thought: I need to find X

Action: search("X")

Observation: result

Thought: now I can answer

Answer: final response

Repeats until goal is reached. Practical challenges: infinite loops without stopping conditions, tool errors need graceful handling, costs can spiral with many steps.

REACT LOOP

AGENTIC PLANNING

LlamaIndex vs LangChain — what's the difference?

ANSWER

LlamaIndex — data framework. Primary strength: ingesting, indexing, retrieving over your own data. 100+ data connectors, sophisticated indexing (knowledge graphs, hierarchical indices). Best for RAG pipelines.

LangChain — workflow framework. Primary strength: chaining LLM calls, agents, multi-step orchestration. Best for complex LLM applications.

Simple rule: data retrieval challenge → LlamaIndex. Workflow orchestration challenge → LangChain. Many production systems use both together.

DATA INGESTION

WORKFLOW ORCHESTRATOR

What is chunking? What strategies do you know?

ANSWER

Splitting documents into smaller pieces before embedding for RAG. One of the most impactful decisions — poor chunking is the most common reason RAG systems fail.

Strategies:

- Fixed-size — split every N tokens, always add overlap
- Recursive character splitting — tries paragraphs → sentences → words. LangChain default.
- Semantic chunking — split where embedding similarity drops. Higher quality, more expensive.
- Structure-aware — split on headers/sections for PDFs, functions for code
- Hierarchical — large chunks for context, small chunks for precise retrieval

Typical parameters: 512-1024 token chunks, 10-20% overlap.

SEMANTIC CHUNKING

RECURSIVE SPLITTING

What is a reranker? Why use one?

ANSWER

A reranker is a cross-encoder model that scores query-document pairs together — more accurate than bi-encoder embedding similarity alone.

Problem it solves: top-K chunks by cosine similarity aren't always the most relevant for answering the specific question.

Pattern: retrieve 20-50 chunks with fast vector search → rerank with cross-encoder → pass top 3-5 to LLM. Popular options: Cohere Rerank, BGE Reranker.

Tradeoff: more accurate but slower than embedding search. Worth it for complex questions where relevance differs from similarity.

CROSS-ENCODER

TWO-STAGE RAG

How do you evaluate a RAG system?

ANSWER

Two dimensions: retrieval quality and generation quality.

Retrieval metrics (RAGAS):

- Context Precision — retrieved chunks that are relevant
- Context Recall — relevant information that was retrieved

Generation metrics (RAGAS):

- Faithfulness — answer grounded in context (hallucination measure)
- Answer Relevance — answer addresses the question
- Answer Correctness — factually correct (needs ground truth)

Evaluation approaches: human evaluation (accurate, doesn't scale), LLM-as-Judge (scalable), ground truth QA datasets. Production monitoring: latency, user feedback, fallback rate, chunk hit rate.

Key lesson: build your evaluation dataset before building the system.

RAGAS METRICS

LLM-AS-A-JUDGE

Walk me through fine-tuning a model.

ANSWER

Step 1 — Define objective: what specific behavior am I improving? This determines everything.

Step 2 — Prepare dataset: instruction-response pairs, minimum 500-5000 examples. Quality beats quantity.

Step 3 — Choose approach: Full fine-tuning (max performance, max compute) or LoRA/QLoRA (parameter efficient, 90% of teams use this).

Step 4 — Train: learning rate $1e-4$ to $2e-4$ for LoRA, 2-5 epochs, using HuggingFace PEFT + Transformers or Axolotl.

Step 5 — Evaluate: compare against base model and prompt-engineered baseline on eval dataset.

Step 6 — Deploy: merge LoRA weights or serve adapter separately.

PEFT TUNING

DATA INGESTION

What is LoRA and QLoRA?

ANSWER

LoRA injects small trainable matrices into frozen attention layers — dramatically fewer trainable parameters.

Technically: for weight matrix W ($d \times d$), adds matrices A ($d \times r$) and B ($r \times d$) where r is rank (typically 4–64). Effective weight = $W + AB$. Only A and B are trained — 10,000x fewer parameters than full fine-tuning.

Benefits: trains on consumer GPUs, adapters are tiny files, multiple adapters swappable on same base model.

QLoRA extends this by quantizing the frozen base model to 4-bit (NF4), training LoRA adapters in 16-bit. Lets you fine-tune 65B models on a single 48GB GPU.

LOW-RANK ADAPTATION

QLORA NORMALFLOAT4

What is quantization?

ANSWER

Reduces numerical precision of model weights — smaller memory, faster inference, some quality tradeoff.

FP32 → 4 bytes/param. FP16/BF16 → 2 bytes. INT8 → 1 byte. INT4 → 0.5 bytes. A 7B model: FP32 = 28GB, INT4 = ~3.5GB.

Methods: GPTQ (post-training, calibration-based), bitsandbytes (INT8/INT4 in Python), GGUF format (llama.cpp, Ollama for local).

Sweet spot for production: INT8 (minimal quality loss). INT4 excellent for local deployment. BF16 preferred for training stability.

QUANTIZATION INT8/INT4

GGUF SERVING

Zero-shot vs one-shot vs few-shot?

ANSWER

Describes how many examples the model gets in the prompt — no weight updates involved.

Zero-shot: just ask. Works for simple, clear tasks. One-shot: one example showing expected format. Few-shot: 2-10 examples. Best for structured tasks, extraction, classification with specific output formats.

Why it works: LLMs have seen enough pretraining data that examples in the prompt activate relevant pattern-matching. You're reminding the model, not teaching it.

Practical tip: for any task where output consistency matters — use few-shot with diverse, representative examples including edge cases.

IN-CONTEXT LEARNING

FEW-SHOT EVALS

What is RLHF?

ANSWER

RLHF — Reinforcement Learning from Human Feedback — transforms a raw pretrained LLM into a helpful, safe assistant.

Three stages:

SFT (Supervised Fine-Tuning) — fine-tune on human-written demonstrations of good behavior.

Reward Model Training — show human raters response pairs, train a model to predict human preferences, outputs scalar quality scores.

PPO Fine-Tuning — use reward model as reward signal in RL loop. LLM generates responses, reward model scores them, LLM updates to score higher.

Result: ChatGPT-style helpfulness. GPT-4, Claude, Gemini all use variants of this.

HUMAN FEEDBACK

PPO RLHF LOOP

What is Constitutional AI?

ANSWER

Anthropic's alignment approach — replaces human feedback with explicit principles (a "constitution") to guide AI behavior.

Process: model generates responses → AI critic evaluates them against the constitution → model revises responses based on critique → revised responses train a preference model.

Key advantage over RLHF: more scalable (no human raters for every example), principles are explicit and auditable, handles harmful content without humans directly labeling it.

Used to train Claude. RLAI (Reinforcement Learning from AI Feedback) is the general category — Constitutional AI is a specific implementation.

[ANTHROPIC CLAUDE](#)[RLAIF PRINCIPLES](#)

Semantic search vs keyword search?

ANSWER

Keyword search (BM25, TF-IDF) — exact term matching. Fast, transparent, precise for known terms, product codes, names.

Semantic search — embedding-based meaning similarity. Handles synonyms, paraphrases, conceptual relationships. "cardiac arrest" finds "heart attack."

Neither is universally better. Best production RAG systems use hybrid search — combine BM25 scores with semantic similarity via Reciprocal Rank Fusion. Consistently outperforms either alone. Weaviate and Qdrant support this natively.

[BM25 KEYWORD](#)[VECTOR SEMANTIC](#)

What is a system prompt? How do you use it?

ANSWER

Instructions provided before the conversation that set model behavior, persona, constraints, and output format for the entire session. Users typically don't see it.

I use it for: defining role ("You are a senior financial analyst"), output format ("Always respond in valid JSON"), constraints ("Never discuss competitor products"), context injection, and safety guardrails.

Practical tips: be specific not vague. Most important instructions first and last (lost in the middle applies here too). Always test against adversarial inputs — users will attempt "ignore all previous instructions."

SYSTEM PROMPT

ADVERSARIAL INPUTS

RAG vs Long Context — when to use each?

ANSWER

Long context (Gemini 1M tokens, Claude 200K) — dump entire knowledge base in context. Simple, no retrieval infrastructure. RAG — retrieve only relevant chunks at query time. More complex, more targeted.

Long context wins when: knowledge base fits comfortably, holistic reasoning across all documents needed, unpredictable query patterns.

RAG wins when: knowledge base too large for any context window, cost matters (fewer tokens = lower cost), freshness matters, citations needed, latency is critical.

As context windows grow, RAG becomes less necessary for small knowledge bases. For enterprise-scale collections — RAG remains essential.

TOKEN LENGTH

CONTEXT STRATEGY

What is GraphRAG?

ANSWER

Standard RAG retrieves chunks by semantic similarity — loses relationship structure between entities. GraphRAG builds a knowledge graph alongside vector embeddings.

Entities and relationships extracted from documents form a graph. Simple factual queries use vector retrieval. Complex relational queries traverse the graph — "what are all subsidiaries of Company X and their performance?" requires connecting information across many documents.

Result: dramatically better on multi-hop reasoning and entity relationship queries. Tradeoff: expensive to build (LLM calls for entity extraction), slower indexing, harder to maintain. Microsoft open-sourced their implementation.

KNOWLEDGE GRAPH

MULTI-HOP SEARCH

What is data poisoning? How do you defend against it?

ANSWER

Adversarial attack corrupting training data to manipulate model behavior — either degrading general performance or implanting backdoor behaviors.

Types:

- Backdoor attacks — model behaves normally except when specific trigger present
- Label flipping — attacker flips correct labels in training data
- Prompt injection (inference-time) — malicious instructions injected into user input

Defenses: data provenance tracking, statistical anomaly detection on training data, sanitization pipelines, differential privacy during training. For LLM apps: input validation, output filtering, system prompt hardening, privilege separation for tool access.

DATA POISONING

THREAT MITIGATION

What is Mixture of Experts (MoE)?

ANSWER

Architecture where only a subset of model parameters activate per token — not all of them.

Structure: N expert networks + a router. For each token, router selects top-2 experts. Only those experts process that token. Others are idle.

Benefits: much larger total capacity without proportional compute cost. GPT-4 rumored ~1.8T parameters, activates ~220B per token. Mixtral 8x7B activates 13B effective parameters from 47B total.

Challenges: load balancing across experts, communication overhead in distributed training, more complex serving infrastructure.

[SPARSE MOE](#)[ROUTING EXPERTS](#)

Design a multi-agent architecture.

ANSWER

My design process:

Decompose the task — break into distinct subtasks that benefit from specialization or parallelization.

Define agent roles — each agent gets a specific role, focused system prompt, and limited relevant tools. Specialization improves performance.

Design orchestration — hierarchical (orchestrator delegates to workers) for most control. Parallel for independent subtasks. Sequential pipeline for linear workflows.

Manage shared state — typed state dictionary all agents read and write (LangGraph's state model).

Handle failures — retry logic, fallback agents, human escalation paths, max iteration limits to prevent infinite loops.

Frameworks: CrewAI for role-based collaboration, AutoGen for conversational multi-agent, LangGraph for maximum workflow control.

MULTI-AGENT

LANGGRAPH STATE

What is prompt injection? How do you defend?

ANSWER

Attack where malicious content in input hijacks LLM behavior by overriding system instructions. Direct: user types "ignore all previous instructions". Indirect: attacker embeds malicious instructions in content LLM retrieves.

Defenses:

- Input validation — filter obvious injection patterns
- Privilege separation — LLM doesn't directly access sensitive operations
- Sandboxed tools — minimal permissions per tool
- Secondary classifier — small model screens inputs before main LLM
- Output validation — check outputs before executing actions
- Instruction hierarchy — clearly separate system vs user content in prompt structure

PROMPT INJECTION

DEFENSE SANDBOX

How does a Transformer generate text?

ANSWER

Autoregressively — one token at a time.

Forward pass → attention across all previous tokens → output logits (one score per vocabulary token) → softmax → apply temperature/Top-K/Top-P → sample one token → append to sequence → repeat.

Key optimization: KV-cache stores Keys and Values from previous tokens so they aren't recomputed each step. Makes generation $O(n)$ instead of $O(n^2)$. Critical for production inference speed.

Masked self-attention ensures each token only attends to previous tokens, not future ones — maintaining the autoregressive property.

ATTENTION KEY/VALUE

PAGEDATTENTION

What is PEFT? Different methods?

ANSWER

Parameter Efficient Fine-Tuning — adapts pretrained models by training only a small fraction of parameters.

Methods:

- LoRA — trainable low-rank matrices in attention layers. Most widely used.
- Prefix Tuning — learnable soft prompt tokens prepended at every layer
- Prompt Tuning — learnable tokens at input embedding layer only. Simpler than prefix tuning.
- Adapter layers — small bottleneck feedforward layers inserted inside each Transformer block
- (IA)³ — scales activations with learned vectors. Minimal parameters.

In practice LoRA has largely won. Achieves competitive performance with 0.1-1% of total parameters trained.

PARAMETER EFFICIENCY

ADAPTER TUNING

What is model distillation?

ANSWER

Training a smaller "student" model to mimic a larger "teacher" model's behavior — achieving similar performance at fraction of size and cost.

Key insight: train student on teacher's soft probability distributions (full output logits), not just hard correct labels. Soft labels carry more information about which wrong answers are more plausible.

When I'd use it: production latency requirements large models can't meet, cost optimization at high volume, edge deployment with size constraints, or when a specialized small model can match a large general model on a specific task.

Examples: DistilBERT is 40% smaller than BERT with 97% of performance. Phi-2 (2.7B) achieves remarkable performance through high-quality distilled training data.

STUDENT-TEACHER

LOGITS MIMICRY

What is the "lost in the middle" problem?

ANSWER

LLMs perform significantly better on information at the beginning and end of long contexts — and substantially worse on information buried in the middle.

For RAG: even if you retrieve the right chunks, if the most relevant chunk is in the middle of a long context, the model may effectively ignore it.

How I handle it: put highest relevance chunk first, second highest last, rest in middle (leverages primacy and recency effects). Retrieve fewer, more targeted chunks. Compress chunks before passing to generator. Use iterative retrieval with focused context per step instead of one large retrieval.

PRIMACY & RECENCY

PROMPT COMPRESSION

How do you ensure reliable structured output from LLMs?

ANSWER

LLMs trained on natural language often produce almost-valid JSON with subtle errors.

Techniques in order of reliability:

- Native function calling / structured output mode (most reliable) — OpenAI, Anthropic support constrained decoding to enforce valid JSON schema.
- Pydantic + instructor library — define schema as Pydantic model, instructor handles retries automatically when output doesn't match.
- Few-shot examples — show 3-5 exact JSON format examples. Dramatically improves consistency.
- Retry with error feedback — catch parse exceptions, send error back to LLM: "your output was invalid because X, fix it." 1-2 retries resolves most failures.
- Constrained decoding (open source) — outlines, guidance libraries enforce grammar at token sampling level.

[STRUCTURED OUTPUTS](#)[PYDANTIC SCHEMAS](#)

What is semantic caching?

ANSWER

Stores previous LLM query-response pairs and returns cached responses for semantically similar future queries — without making new API calls.

Unlike standard caching (exact match only), semantic caching embeds incoming queries, searches cache vector store, returns cached response if similarity exceeds threshold.

Cost impact: significant for applications with repetitive patterns — customer service, FAQ bots, internal knowledge bases. If 40% of queries are semantically equivalent to previous ones, eliminate 40% of API costs.

Implementation: GPTCache (standard library), Redis with vector search. Key parameter: similarity threshold — tune carefully to avoid returning wrong cached answers.

[SEMANTIC CACHING](#)[GPTCACHE / REDIS](#)

What is RLAIIF?

ANSWER

Reinforcement Learning from AI Feedback — replaces human raters in RLHF with an AI model doing the evaluation.

Constitutional AI is Anthropic's specific RLAIIF implementation: generate responses → AI critic evaluates against explicit principles → AI-generated preferences train reward model → RL fine-tuning with that reward model.

Why it matters: scales better than human annotation, principles are explicit and auditable. The field is moving toward RLAIIF because human annotation is the bottleneck in alignment research at scale.

[RLAIIF FEEDBACK](#)[CONSTITUTIONAL AI](#)

How do you handle long documents where chunks lose context?

ANSWER

Common problem — individual chunks often lack context to be meaningful alone.

Strategies:

- Contextual retrieval — use LLM to generate brief context description for each chunk before embedding, prepend to chunk. Anthropic published this — dramatically improves retrieval accuracy.
- Parent document retrieval — small chunks for precise retrieval, return parent larger chunk to LLM. Precise retrieval, rich context.
- Sentence window retrieval — embed at sentence level, return surrounding sentences (± 3) to LLM.
- Metadata enrichment — attach document title, section header, date to each chunk. Include in text before embedding.
- Summary index — hierarchical: document summary nodes + chunk nodes. Broad queries hit summaries, specific queries hit chunks.

METADATA CONTEXT

ANTHROPIC RETRIEVAL

How do you build an evaluation dataset for LLMs?

ANSWER

Step 1 — Define task coverage: what question types will real users ask? Factual, multi-hop, summarization, comparison? Sample from each.

Step 2 — Collect QA pairs: from real user queries if available. Otherwise synthetic — prompt LLM with documents to generate diverse questions.

Step 3 — Add ground truth: correct answer + source document(s) for each question.

Step 4 — Include adversarial examples: unanswerable questions (model should say "I don't know"), ambiguous queries, multi-document synthesis questions.

Step 5 — Size: 100-500 examples typically sufficient for RAG evaluation.

Most important lesson: build the evaluation dataset before building the system. Every architectural decision should be measured against it.

[RAG EVALUATION](#)[TEST DATASETS](#)

What is Chain of Thought prompting? When does it help?

ANSWER

CoT asks the model to show reasoning steps before final answer. Forces externalization of intermediate computation.

Helps significantly: multi-step math, logical reasoning, planning tasks, anything where answer depends on intermediate steps.

Doesn't help or hurts: simple factual retrieval, tasks where model's reasoning is confidently wrong (CoT just elaborates the error), high-speed low-latency applications (longer output = higher cost).

Variants: Self-consistency CoT — generate multiple chains, take majority answer. Tree of Thoughts — explore multiple reasoning branches. Both improve reliability over single CoT for complex problems.

[REASONING CHAIN](#)[TREE OF THOUGHTS](#)

Open source vs closed source LLMs — when do you choose each?

ANSWER

Closed source (GPT-4, Claude, Gemini): higher capability on complex reasoning, managed infrastructure, no serving complexity. Downsides: data leaves your infrastructure, API costs scale with usage, model behavior can change without notice.

Open source (LLaMA 3, Mistral, Qwen): full data control, fine-tunable, cheaper at high volume, no rate limits, model version stability. Downsides: requires ML infrastructure expertise, generally lower capability (gap narrowing fast).

My decision: prototype and low volume → closed source for speed. High volume, sensitive data (healthcare/finance/legal), compliance requirements, or fine-tuning needed → open source. Cost savings of self-hosted at scale are often compelling.

[OPEN VS CLOSED](#)[DEPLOY COST](#)

What is MMLU and how are LLMs benchmarked?

ANSWER

MMLU — Massive Multitask Language Understanding — 57 subjects from elementary to professional level. One of the most cited benchmarks.

Other key benchmarks:

- HumanEval/MBPP — code generation
- GSM8K — grade school math reasoning
- TruthfulQA — measures truthfulness vs plausible-sounding falsehoods
- MT-Bench — multi-turn conversation, GPT-4 as judge
- LMSYS Chatbot Arena — real human preference voting. Most reliable real-world signal.

Important caveat: benchmark performance $\neq$ production performance. Models can be fine-tuned on benchmark data (contamination). Always evaluate on your specific use case with your own evaluation dataset before choosing a production model.

[MMLU BENCHMARKS](#)[CHATBOT ARENA](#)

How do you reduce hallucinations in production?

ANSWER

Layered defense — no single technique eliminates them.

Layer 1 — RAG: ground responses in retrieved verified context. Instruct model to say "I don't have information about this" when context doesn't contain the answer.

Layer 2 — Prompt mitigations: lower temperature, explicit uncertainty instructions, ask for citations per claim.

Layer 3 — Consistency checking: generate multiple responses, flag inconsistent ones.

Layer 4 — Output validation: NLI model checks if response claims are entailed by source documents.

Layer 5 — LLM-as-Judge: separate LLM evaluates faithfulness, flags low-confidence responses.

Layer 6 — Human escalation: route low-confidence responses to human review for high-stakes applications.

Honest answer: hallucinations cannot be completely eliminated. Goal is detection and mitigation, not elimination.

RELIABLE OUTPUTS

HALLUCINATION GUARD

What is model drift? How do you monitor LLMs in production?

ANSWER

Two forms: data drift (user query patterns change) and model drift (underlying model changes — especially closed source APIs updated by providers).

What I monitor:

Input: query length distribution, topic distribution, out-of-domain query rate.

Output: response length, refusal rate, latency. Sudden changes indicate problems.

Quality: sample 5% of production queries, evaluate with LLM-as-Judge against quality rubrics, track scores over time, alert on degradation.

RAG-specific: context relevance scores, retrieval latency, fallback rate.

User signals: thumbs up/down, conversation abandonment, follow-up clarification questions.

Regression testing: run eval dataset against production regularly. For closed source API users — always test new model versions against your eval dataset before migrating.

MODEL DRIFT

PROMETHEUS METRICS

Design a production RAG system for a company's internal knowledge base.

ANSWER

Requirements I'd clarify first: document count, formats, update frequency, concurrent users, latency requirements, compliance constraints. Assuming 100K documents, mixed formats, weekly updates, 1000 concurrent users, sub-3 second response, enterprise compliance.

Ingestion pipeline: format-specific loaders → text extraction → metadata enrichment (title, author, date, department) → recursive chunking at 512 tokens with 50 token overlap → contextual chunk enrichment → embed with text-embedding-3-small → index in Qdrant (self-hosted for compliance, supports hybrid search) → Airflow for scheduled reindexing.

Query pipeline: multi-query generation (3 variants) → hybrid retrieval (semantic + BM25, RRF fusion) → retrieve top 20 → rerank to top 5 → construct prompt → LLM generation → citation extraction.

Evaluation: RAGAS eval suite offline, LLM-as-Judge sampling 5% of production queries, Langsmith for tracing.

Infrastructure: FastAPI backend, Redis semantic cache (30% cost reduction target), Kubernetes for scaling, Prometheus and Grafana for metrics.

Guardrails: input validation against injection, PII detection before external API calls, output filtering, human escalation for low-confidence responses.

SYSTEM DESIGN

ENTERPRISE RAG



Ready to Ace Your GenAI Interview?

These 50 questions cover the critical technical, system design, and production concepts expected of AI Engineers today. Master the details, focus on trade-offs (latency, cost, accuracy), and show structured engineering thinking in your answers.



Follow ResnurAI

🌱 One critical GenAI topic explained simply. Every single day.



Save this Post

📌 Save this — review it immediately before your next loop.



Share & Repost

🔄 Repost to share these questions with your engineering network.

